Master d'informatique – 1 ***Architecture des Systèmes Intégrés***

December 2009 ***Documents allowed - Duration 2h***

It is imperative to use the provided squared sheets for the simplified diagrams

The aim of this exercise is to compare the execution performance of a sort algorithm on two different Mips implementations. The first is the classical realization on a 5-stage pipeline presented in class. This realization is called **Mips**. The second, called **SS2**, is the superscalar 5-stage two-pipeline realization presented in this course. **SS2** has the same clock frequency as the **Mips** realization.

One of the most powerful sort algorithms is QuickSort. This algorithm is applied to an array of integers. The heart of this algorithm is to find the definitive place of the first element of the array (called the pivot), in the sorted array, and to place it. Elements larger than the pivot are placed after it, and smaller elements before it.

```
unsigned int PlacePivot (int *tab, unsigned int size)
  {
  unsigned int idx   = 1;
  int          pivot = 0;
  int          tmp   = 0;
  unsigned int i     = 1;

  pivot = tab [0];
  for (i=1 ; i!=size ; i++)
     {
    if (tab [i] < pivot)
      {
      tmp       = tab [i  ];
      tab [idx] = tmp      ;
      tab [i  ] = tab [idx];
      idx ++;
      }
    }
  tab [0    ] = tab [idx-1];
  tab [idx-1] = pivot      ;

  return (idx);
  }
```

Two versions of the main loop of this function have been produced by a Mips pipeline assembler. On the loop entry, register R4 contains the address of second element of the array (the address of **`tab[1]`**). Register R5 contains the end-of-array address, register R8 the pivot value and register R9 the address of the element of index **`idx`**.

1st version :

```
Loop1 :          Lw      r10, 0 (r4 )       ; tab [i]
                 Slt     r11, r10, r8       ; tab [i] < pivot

                 Beq     r11, r0 , Endif1
                 Nop
                 Lw      r12, 0 (r9 )       ; tab [idx]
                 Sw      r12, 0 (r4 )
                 Sw      r10, 0 (r9 )
                 Addiu   r9 , r9 , 4

Endif1 :         Addiu   r4 , r4 , 4
                 Bne     r4 , r5 , Loop1
                 Nop
```

2nd version :

```
Loop2 :     Lw      r10, 0 (r4 )        ; tab [i  ]
            Lw      r12, 0 (r9 )        ; tab [idx]
            Sw      r12, 0 (r4 )
            Sw      r10, 0 (r9 )
            Slt     r11, r10, r8        ; tab [i] < pivot
            Sll     r11, r11, 2
            Addu    r9 , r9 , r11

            Addiu   r4 , r4 , 4
            Bne     r4 , r5 , Loop2
            Nop
```

1- Analyze the execution of the **Loop1** loop on the **Mips** realization in the case where the **Beq** branch succeeds and in the case where it fails. You are not asked to draw a diagram, but simply to indicate the data dependencies and situations that cause stall cycles. Calculate the average number of cycles per iteration, assuming that in 50% of cases the branch succeeds. Calculate the CPI and CPI-useful.

2- Analyze the execution of the **Loop1** loop using a simplified diagram on the **SS2** realization in the case where the **Beq** branch succeeds and in the case where it fails. It is assumed that the **Loop1** label is placed on an aligned address and that the instruction buffer is empty at the entry of the loop. Calculate the average number of cycles per iteration, assuming that 50% of the time the branch succeeds. Calculate the CPI and CPI-useful.

3- Analyze the execution of the **Loop2** loop on the **Mips** implementation. You are not asked to draw a diagram, but simply to indicate the data dependencies and situations that cause stall cycles. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.

4- Analyze the execution of the **Loop2** loop using a simplified diagram on the **SS2** implementation. Assume that the **Loop2** label is placed on an aligned address and that the instruction buffer is empty at the entry of the loop. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.

5- For the **Mips** realization, reorder the **Loop1** code to avoid lost cycles as much as possible. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful under the same hypothesis (50% of branches succeeds).

6- For the **Mips** implementation, reorder the **Loop2** code to avoid lost cycles as much as possible. Calculate the average number of cycles per iteration. Calculate CPI and CPI-useful.

7- For **Mips**, optimize the **Loop1** code using the "software pipeline" technique, clearly indicating which instructions belong to which stage. Then reorder to avoid lost cycles as much as possible. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.

8- For **SS2**, unroll the **Loop2** code once (processing two elements per iteration) and optimize the code to avoid lost cycles as much as possible.

From this point, we consider the original code of both versions. We are interested in the **Mips** implementation.

The processor is connected to the memory via a data cache and an instruction cache. We want to study the influence of the data cache on the execution performance. The instruction cache is assumed to be perfect (Miss rate = 0).

The data cache is a direct-mapping write-through cache. It has a capacity of 16 Kbytes. A cache block contains 32 bytes.

In case of a hit, 1 cycle is required to access the cache.

In case of a miss, it takes 1 cycle to detect the miss. Then, on average, it takes 3 cycles to initialize the memory access. Once memory access has been initialized, one word can be transferred per cycle. When the block has been fully received, it takes 1 cycle to update the cache memories and a further 1 cycle to respond to the processor.

First, we take into account the impact of loads, neglecting the effect of stores.

At the entry of the loop, the cache is assumed to be empty. The array to be sorted contains 1 K integers and the array address is aligned (multiple of 4096).

9- A cache block contains how many words. In the case of a miss, how many cycles are required for a load ?

10- For each of the two versions, how many Misses occur during the execution of the iterations.

11- Calculate the CPI-useful of the loop for each of the two versions.

12- How does the miss rate change if we replace the direct-mapping Write Through cache with a direct-mapping Write Back cache ? Justify your answer.

13- How does the miss rate change if we replace the direct-mapping Write Through cache with an set associative Write Through cache and increase the associativity level ? Justify your answer.

We are now interested in the effect of stores.

The Write Through direct-mapping data cache contains a one-place write buffer. From the processor's point of view, a write can only take place if there is a place in the buffer. In this case, writing to the buffer requires 1 cycle. When the buffer contains a write request, the cache takes an average of 4 cycles to write to memory and empty the buffer.

14- For each of the two versions, calculate the useful CPI of the loop, taking into account the effect of loads and stores.

15- Keeping a Write Through cache, how the cache may be modified to reduce the impact of stores.

16- Calculate the CPI-useful if we implement the solution you propose.